

身份驗證

- **13-1: 登入與登出**
 - 註冊與登入使用PasswordHasher 13-2
 - 登出 13-8
- **13-2: 權限設定**
 - 授權保護頁面使用Authorize 13-9
 - 身分識別的要點 13-11
- **13-3: ASP.NET Core Identity**
 - ASP.NET Core Identity簡介 13-12
 - Claim / Session / Cookie 比較 13-15

Module 13

註冊與登入使用 PasswordHasher -1

- 登入的流程與需要的資料表欄位：

使用者輸入帳密



查詢使用者資料 (資料庫)



驗證密碼雜湊



建立 Claims



簽發 Authentication Cookie



後續 Request 自動通過授權

欄位	說明
Id	使用者識別
Email	登入帳號
PasswordHash	密碼雜湊
Role	使用者角色

註冊與登入使用 PasswordHasher -2

- 使用者註冊或設定密碼時加入雜湊：

```
var hasher = new PasswordHasher<User>();  
user.PasswordHash = hasher.HashPassword(user, rawPassword);
```

- 登入時驗證比對雜湊：
- PasswordHasher** 幫你處理 salt、迭代與版本升級。
- 註冊用 **HashPassword**，登入用 **VerifyHashedPassword**。

```
var hasher = new PasswordHasher<User>();  
  
var result = hasher.VerifyHashedPassword(  
    user,  
    user.PasswordHash,  
    inputPassword);  
  
if (result == PasswordVerificationResult.Failed)  
{  
    // 密碼錯誤  
}
```

註冊與登入使用 PasswordHasher -3

- 登入：驗證 PasswordHash (Verify Password)：

- PasswordHasher 幫你處理 salt、迭代與版本升級。

- 註冊用 HashPassword，登入用 VerifyHashedPassword。

```
using Microsoft.AspNetCore.Identity;

public async Task<bool> LoginAsync(string email, string inputPassword)
{
    var user = await _db.Users.SingleOrDefaultAsync(x => x.Email == email);
    if (user == null) return false;

    var hasher = new PasswordHasher<User>();

    var result = hasher.VerifyHashedPassword(
        user,
        user.PasswordHash,
        inputPassword
    );

    if (result == PasswordVerificationResult.Failed)
        return false;

    // ✅ 如果 result == SuccessRehashNeeded，代表雜湊參數較舊，建議升級（見下一節）
    return true;
}
```

註冊與登入使用PasswordHasher -4

- 建立 Claims 並簽發 Cookie :
- 請加入AntiForgeryToken標籤，防止CSRF(借用Cookie攻擊網站)。

```
@Html.AntiForgeryToken()
```

```
[ValidateAntiForgeryToken]  
public IActionResult Login(...) { }
```

```
[HttpPost]  
public async Task Login(string email, string password)  
{  
    var user = _db.Users.FirstOrDefault(u => u.Email == email);  
    if (user == null) return Unauthorized();  
  
    var hasher = new PasswordHasher<User>();  
    var verify = hasher.VerifyHashedPassword(user, user.PasswordHash, password);  
  
    if (verify == PasswordVerificationResult.Failed)  
        return Unauthorized();  
  
    var claims = new List<Claim>  
    {  
        new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),  
        new Claim(ClaimTypes.Name, user.Email),  
        new Claim(ClaimTypes.Role, user.Role)  
    };  
  
    var identity = new ClaimsIdentity(claims, "MyCookie");  
    var principal = new ClaimsPrincipal(identity);  
  
    await HttpContext.SignInAsync("MyCookie", principal);  
  
    return RedirectToAction("Index", "Home");  
}
```

註冊與登入使用 PasswordHasher -5

- “MyCookie”是自訂的Cookie名稱，可依需求自訂，或使用官方提供的CookieAuthenticationDefaults.AuthenticationScheme，幫你管理好cookie名稱。

```
if (verify == PasswordVerificationResult.Failed)
    return Unauthorized();

var claims = new List<Claim>
{
    new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
    new Claim(ClaimTypes.Name, user.Email),
    new Claim(ClaimTypes.Role, user.Role)
};

var identity = new ClaimsIdentity(claims, "MyCookie");
var principal = new ClaimsPrincipal(identity);

await HttpContext.SignInAsync("MyCookie", principal);
```

註冊與登入使用 PasswordHasher -6

- 設定 Cookie Authentication (Program.cs) :

```
builder.Services
    .AddAuthentication("MyCookie")
    .AddCookie("MyCookie", options =>
    {
        options.LoginPath = "/Account/Login";
        options.AccessDeniedPath = "/Account/Denied";
        options.Cookie.HttpOnly = true;
        options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
        options.Cookie.SameSite = SameSiteMode.Lax;
    });

builder.Services.AddAuthorization();
```

- 注意：要設定在app.UseAuthorization();之前。

登出

- 將簽發的 **Cookie** 清除：

```
public async Task<IActionResult> Logout()  
{  
    await HttpContext.SignOutAsync("MyCookie");  
    return RedirectToAction("Login");  
}
```


授權保護頁面使用Authorize -1

- ClaimTypes中的Role是用來設定登入者的權限，以字串儲存。

```
if (verify == PasswordVerificationResult.Failed)
    return Unauthorized();

var claims = new List<Claim>
{
    new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
    new Claim(ClaimTypes.Name, user.Email),
    new Claim(ClaimTypes.Role, user.Role)
};

var identity = new ClaimsIdentity(claims, "MyCookie");
var principal = new ClaimsPrincipal(identity);

await HttpContext.SignInAsync("MyCookie", principal);
```

授權保護頁面使用Authorize -2

- 以標籤控制授權頁面。
- **Controller :**

```
[Authorize]
public class MemberController : Controller
{
    public IActionResult Index() => View();
}
```

- **角色限制 :**

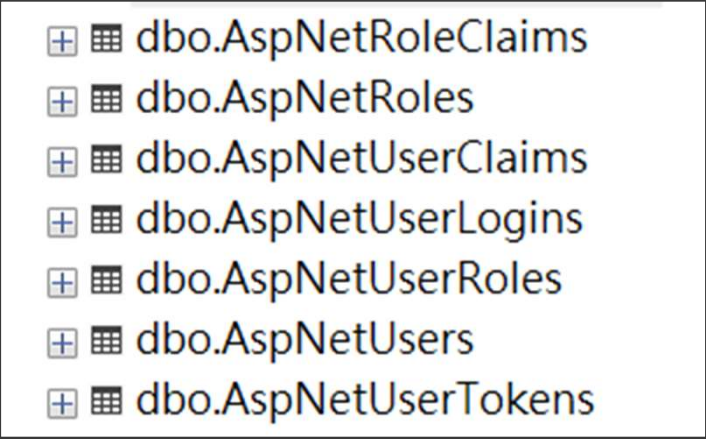
```
[Authorize(Roles = "Admin")]
public IActionResult AdminOnly()
{
    return View();
}
```

身分識別的要點

- **Cookie** 只存 **Claims** (身分識別) 。
- 密碼永遠只存雜湊 。
- 授權不是「有登入就好」 。
- 重要操作一定要檢查角色或 **Policy** 。

ASP.NET Core Identity簡介-1

- Identity是官方提供的驗證框架。
- 先前的製作都是使用Identity中的工具。
- 預選「身份驗證」的專案，在建立後需先用以下指令將Identity所需的資料庫及Table建起：
 - dotnet tool install --global dotnet-ef
 - dotnet ef migrations add InitialIdentity (可自訂)
 - dotnet ef database update

A screenshot of a database table list, likely from SQL Server Enterprise Manager. It shows a list of tables in the 'dbo' schema. Each entry has a small icon to its left, consisting of a plus sign in a square followed by a grid icon. The tables listed are: dbo.AspNetRoleClaims, dbo.AspNetRoles, dbo.AspNetUserClaims, dbo.AspNetUserLogins, dbo.AspNetUserRoles, dbo.AspNetUsers, and dbo.AspNetUserTokens.

+	db	dbo.AspNetRoleClaims
+	db	dbo.AspNetRoles
+	db	dbo.AspNetUserClaims
+	db	dbo.AspNetUserLogins
+	db	dbo.AspNetUserRoles
+	db	dbo.AspNetUsers
+	db	dbo.AspNetUserTokens

ASP.NET Core Identity 簡介-2

- 在前一個 Lab 中，我們沒有從頭使用整套 Identity，但已經完成了：
 - 使用者註冊
 - 密碼雜湊 (PasswordHasher)
 - Cookie 驗證
 - Claims 建立
 - 角色與授權控制
- 換句話說，我們已經完整實作了一套身分驗證機制。

手動做過一次之後：

- 你知道 Cookie 是何時產生的
- 你知道 Claims 是怎麼放進去的
- 你知道授權判斷發生在哪

這時再看 Identity，會很清楚：

「原來它幫我做的是這些。」

ASP.NET Core Identity簡介-3

- 整包的 Identity 適合什麼時候用？
- 適合：
 - 新專案
 - 標準會員系統
 - 不想自行維護安全細節
- 不一定適合：
 - 已有既定使用者資料表
 - 權限模型高度客製
 - 需與舊系統整合

Claim / Session / Cookie 比較 -1

項目	Claim	Session	Cookie
本質	使用者身分與權限資訊	伺服器端狀態儲存機制	瀏覽器端儲存機制
存放位置	Authentication Ticket (通常在 Cookie 內)	Server 記憶體 / Cache / DB	Client (瀏覽器)
是否跨 Request	✅ 是	✅ 是	✅ 是
是否跨 Server	✅ 是 (若使用 Cookie/JWT)	❌ 預設不行	✅ 是
常存內容	UserId、Role、權限	購物車、暫存資料	SessionId、登入憑證
是否可被使用者看到	❌ (加密/簽章)	❌	⚠️ 可看到但不可改
是否適合身分驗證	✅ 非常適合	⚠️ 不建議	⚠️ 本身不是
ASP.NET Core 常用性	★★★★★	★★	★★★★

Claim / Session / Cookie 比較 -2

- **Cookie**：瀏覽器幫你保存的小資料
 - **Session**：伺服器幫你記住狀態
 - **Claim**：描述「你是誰、你能做什麼」
-
- **Cookie** 是載體，**Session** 是伺服器狀態，**Claim** 是身分與權限的事實描述。

Claim / Session / Cookie 比較 -3

- 現代常見做法：

場景	作法
Web MVC	Cookie + Claims
Web API	JWT + Claims
分散式系統	SSO / Token
短期狀態	少量 Session / Cache